

Chapter 2 — DTOs and Zod

(Services/crud.dtos.ts)

File: `src/features/react-crud-v4/Services/crud.dtos.ts` **One job:** Define the exact shape of every API request and response. Validate responses at runtime with Zod.

2.1 What is a DTO?

DTO = Data Transfer Object. It is the exact shape of data that travels between the frontend and backend.

Think of it as a contract: "The backend promises to send data in this shape. The frontend promises to send requests in this shape."

Why does this matter? Without a contract:

- The backend changes a field name from `StudentName` to `Name` — your app silently breaks
- The backend starts returning `null` for a field that used to always have a value — your app crashes in a confusing way
- A new developer adds a field to the frontend request that the backend doesn't expect — the request silently fails

With a DTO contract:

- TypeScript catches mismatches at compile time
 - Zod catches unexpected runtime data at the API boundary with a clear error message
-

2.2 Why TypeScript Alone Is Not Enough

This is the most important concept in this chapter. Read it carefully.

TypeScript is a compile-time tool. When you run `npm run build`, TypeScript checks your code and then *disappears*. The browser runs plain JavaScript — TypeScript types do not exist at runtime.

```
// You write this TypeScript:  
type ItemDto = { StudentName: string; StudentNumber: number; };  
  
// At runtime, the browser runs this JavaScript:  
// (no types – they are erased)
```

The problem: When your app fetches data from the server, the server sends JSON. TypeScript cannot check that JSON at runtime. If the server sends:

```
{ "StudentName": null, "StudentNumber": "not-a-number" }
```

TypeScript will NOT warn you. Your `ItemDto` type says `StudentName: string` but the actual value is `null`. TypeScript trusts you that the data matches the type — it cannot verify it.

What happens next? The `null` travels silently through your app until some code calls `.toUpperCase()` on it and crashes with: `"Cannot read properties of null (reading 'toUpperCase')"`. This error is 5 function calls away from the actual problem (the API returning null). You spend 30 minutes debugging.

Zod solves this. Zod validates the *actual data* at the moment it arrives from the server. If the data doesn't match the schema, Zod throws immediately with a clear error: `"Expected string, received null at path: StudentName at getItem"`. You fix it in 2 minutes.

2.3 The Real File — Line by Line

Here is the actual `crud.dtos.ts` from this codebase:

```
import { z } from "zod";
```

`import { z } from "zod"` — imports Zod's main object. Everything in Zod is accessed through `z`. `z.string()`, `z.number()`, `z.object()`, `z.array()` — all of these are methods on `z`.

The Item DTO Schema

```
export const ItemDtoSchema = z.object({
  StudentName: z.string(),
  StudentKey: z.string(),
  CurrentYear: z.string(),
  StudentNumber: z.number(),
  NSN: z.string(),
  HomeClass: z.string(),
  CoreClass: z.string(),
  House: z.string(),
  HomeRoom: z.string(),
  HomeTeacher: z.string(),
  Dean: z.string(),
  Deputy: z.string(),
  PreviousTermStatusId: z.number(),
  CurrentTermStatusId: z.number(),
  LastThresholdCrossedDate: z.string().nullable(),
  DaysAbsentT1: z.number(),
  DaysAbsentT2: z.number(),
  DaysAbsentT3: z.number(),
  DaysAbsentT4: z.number(),
  DaysAbsentYtd: z.number(),
  LastActivityDate: z.string().nullable(),
  LastActivityId: z.number(),
  LastActivityName: z.string(),
  CurrentTermStatusName: z.string(),
```

```

    CurrentTermStatusColor: z.string(),
    PreviousTermStatusName: z.string(),
    PreviousTermStatusColor: z.string(),
  });

```

export const ItemDtoSchema — **export** makes it available to other files. **const** means it never changes. **ItemDtoSchema** is the name — by convention, schemas are named **[Thing]Schema**.

z.object({ ... }) — creates a schema that validates a JavaScript object. The keys inside must match the object's keys exactly. Extra keys are stripped by default (Zod ignores fields the schema doesn't know about — this is safe and intentional).

z.string() — validates that the value is a string. If the value is **null**, **undefined**, **42**, or anything else, Zod throws.

z.number() — validates that the value is a number.

z.string().nullable() — validates that the value is either a string OR **null**. This is important: **LastThresholdCrossedDate** and **LastActivityDate** can be **null** when a student has no activity. Without **.nullable()**, Zod would throw when the server sends **null** for these fields.

Why PascalCase field names? The C# backend uses PascalCase by convention (**StudentName**, not **studentName**). The DTO must match exactly what the server sends. We rename to camelCase in the mapper — not here.

Best practice: The DTO is a 1:1 mirror of the backend model. Never add UI concerns (formatted dates, display names) to the DTO. Keep it pure.

The Response Schema

```

export const GetItemsResponseSchema = z.array(ItemDtoSchema);

```

z.array(ItemDtoSchema) — validates that the value is an array where every element matches **ItemDtoSchema**. If the server sends a single object instead of an array, Zod throws. If any element in the array has a missing or wrong-typed field, Zod throws with the index and path of the problem.

Why a separate schema for the response? Some APIs return a bare array (**[{...}, {...}]**). Others return a wrapper object (**{ Items: [{...}, {...}], Total: 50 }**). The response schema captures this structure. In this case, the NZAMP API returns a bare array, so **z.array(ItemDtoSchema)** is correct.

Inferred Types

```

export type GetItemsResponse = z.infer<typeof GetItemsResponseSchema>;
export type ItemDto = z.infer<typeof ItemDtoSchema>;

```

`z.infer<typeof ItemDtoSchema>` — this is the most important pattern in the file. Let's break it down:

- `typeof ItemDtoSchema` — in TypeScript, `typeof` on a variable gives you the TypeScript type of that variable. `ItemDtoSchema` is a Zod schema object, so `typeof ItemDtoSchema` is the Zod schema's TypeScript type.
- `z.infer<...>` — Zod's generic utility that extracts the *output type* of a schema. It reads the schema definition and produces the equivalent TypeScript type.

The result: `ItemDto` is automatically:

```
type ItemDto = {
  StudentName: string;
  StudentKey: string;
  CurrentYear: string;
  StudentNumber: number;
  // ... all other fields
  LastThresholdCrossedDate: string | null; // because .nullable()
  LastActivityDate: string | null;         // because .nullable()
}
```

Why not write this type manually? Because then you have two sources of truth that can drift apart:

```
// BAD — two sources of truth
const ItemDtoSchema = z.object({ StudentName: z.string() });
type ItemDto = { StudentName: string }; // duplicated — can drift

// If you add a field to the schema but forget the type:
const ItemDtoSchema = z.object({ StudentName: z.string(), NewField: z.number() });
type ItemDto = { StudentName: string }; // WRONG — missing NewField
// TypeScript won't catch this because the type and schema are separate
```

```
// GOOD — single source of truth
const ItemDtoSchema = z.object({ StudentName: z.string() });
type ItemDto = z.infer<typeof ItemDtoSchema>; // always in sync

// Add a field to the schema:
const ItemDtoSchema = z.object({ StudentName: z.string(), NewField: z.number() });
type ItemDto = z.infer<typeof ItemDtoSchema>; // automatically gains NewField ✓
```

Category DTO (Nested Object Response)

```
export const GetCategoriesResponseSchema = z.object({
  ResponseActivities: z.array(CategoryDtoSchema),
});
```

```
ResponseActivityGroups: z.array(CategoryGroupDtoSchema),
});
```

This shows a different response shape — the API returns a wrapper object with two arrays inside. Compare with `GetItemsResponseSchema` which is a bare `z.array(...)`. Always check the C# controller's return type to know which shape to use.

Years DTO (Nested Array in Object)

```
export const GetYearsRetrieveResponseSchema = z.object({
  YearList: z.array(YearDataSchema),
});
```

The years API returns `{ YearList: [...] }`. To access the years in the hook:

```
yearsResponse?.YearList.map((y) => y.Year)
```

Not `yearsResponse?.map(...)` — because the array is nested inside `YearList`.

Request Types (Plain TypeScript, No Zod)

```
export type CreateItemRequest = {
  studentIds: number[];
  selectedStudents: { studentId: number; studentKey: string }[];
  activityId: number;
  completedDate: Date;
  completedBy: number;
  notes: string;
  restrictedNotes: boolean;
  tempAttachments: [];
};
```

Why plain `type` here (no Zod schema)? Request bodies are data *we construct* — we control every field. TypeScript's compile-time checking is sufficient. Zod is only needed for data arriving *from* external sources we don't control.

`studentIds: number[]` — an array of numbers. TypeScript syntax for arrays: `Type[]` or `Array<Type>`.

`selectedStudents: { studentId: number; studentKey: string }[]` — an inline object type inside an array. This is valid TypeScript. For complex nested types, you might extract them to a named type.

`tempAttachments: []` — an empty tuple type. This field is required by the API but always sent as an empty array. The `[]` type means "an empty array" — TypeScript will enforce that you never put anything in it.

2.4 Zod Cheat Sheet

The most common Zod validators used in this codebase:

Validator	What it accepts	Example
<code>z.string()</code>	Any string	"hello", ""
<code>z.number()</code>	Any number	42, 3.14, 0
<code>z.boolean()</code>	true or false	
<code>z.date()</code>	JavaScript Date object	
<code>z.string().nullable()</code>	String or null	"hello", null
<code>z.string().optional()</code>	String or undefined	"hello", undefined
<code>z.string().nullish()</code>	String, null, or undefined	
<code>z.string().min(1)</code>	Non-empty string	
<code>z.string().email()</code>	Valid email format	
<code>z.number().min(1)</code>	Number ≥ 1	
<code>z.array(schema)</code>	Array of items matching schema	
<code>z.object({ ... })</code>	Object with matching keys	
<code>z.string().default("")</code>	String, defaults to "" if absent	
<code>z.string().refine(fn, msg)</code>	String passing custom function	

2.5 Common Mistakes

✗ Forgetting `.nullable()` for fields that can be null

```
// WRONG – crashes when server sends null
LastActivityDate: z.string(),

// CORRECT – accepts null
LastActivityDate: z.string().nullable(),
```

How to know which fields can be null: Check the C# model. If the C# property is `string?` (nullable reference type) or `DateTime?` (nullable value type), add `.nullable()` to the Zod schema.

✗ Writing types manually instead of using `z.infer`

```
// WRONG – can drift from schema
const ItemDtoSchema = z.object({ StudentName: z.string() });
```

```
type ItemDto = { StudentName: string }; // duplicated

// CORRECT
type ItemDto = z.infer<typeof ItemDtoSchema>;
```

✗ Adding UI concerns to the DTO

```
// WRONG – DTO has formatted date (UI concern)
export const ItemDtoSchema = z.object({
  LastActivityDate: z.string().nullable().transform(d => d ?
dayjs(d).format("DD/MM/YYYY") : null),
});

// CORRECT – DTO is 1:1 with backend. Formatting happens in the mapper.
export const ItemDtoSchema = z.object({
  LastActivityDate: z.string().nullable(),
});
```

✗ Using Zod for request bodies

```
// WRONG – unnecessary, we control request data
export const CreateItemRequestSchema = z.object({ ... });
export type CreateItemRequest = z.infer<typeof CreateItemRequestSchema>;

// CORRECT – plain TypeScript type is sufficient
export type CreateItemRequest = { ... };
```

2.6 How to Add a New Field to the DTO

When the backend adds a new field to the API response:

1. Add it to the Zod schema in `crud.dtos.ts`:

```
NewField: z.string(),
```

2. The TypeScript type (`ItemDto`) automatically gains the field via `z.infer`.
3. Add it to `ItemRow` in `mapItemDtoToRow.ts`.
4. Map it in `mapItemDtoToRow`:

```
newField: dto.NewField,
```

5. Add a column in `itemColumns.tsx` if it should be displayed.

That's it. TypeScript will guide you — if you add the field to the schema but forget the mapper, TypeScript will show an error on the mapper function saying `newField` is missing from the return object.

Next: [Chapter 3 — Types and Forms](#)